COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010 - Data Structures and Algorithms	**SECTION:**	CPE21S3
DATE PERFORMED:	July 09, 2026	**DATE SUBMITTED:**	July 12, 2026
NAME:	Garbosa, Isaiah Benedict R.	**INSTRUCTOR:**	Engr. Jimlord Quejado

Hands-on Activity 1.1 Basic C++ Programming

1. PROCEDURE OUTPUT

ILO A: Create Code That Follows the Basic C++ Code Structure

- Create a function that will take two numbers and display the sum.
- Create a function that will return whether variable A is greater than variable B.
- Create a function that will take two Boolean values and display the result of all logical operations then return true if it was a success.

ILO_A.cpp

```cpp
#include <iostream>


int sumOfTwoNum(int fNum, int sNum);
bool greaterThan(int numA, int numB);
bool operations(int x, int y);


int main()
{


    int fNum, sNum, numA, numB, x, y;


    //Create a function that will take two numbers and display the sum.


    std::cout << "Enter first number: " << std::endl;
    std::cin >> fNum;


    std::cout << "Enter second number: " << std::endl;
    std::cin >> sNum;


    std::cout << "\nThe sum of the two number is "<<sumOfTwoNum(fNum,sNum)<< std::endl;


    //Create a function that will return whether variable A is greater than variable B.
    std::cout << "\nEnter first number: " << std::endl;
    std::cin >> numA;
```

```
33
34
35      std::cout << "Enter second number: " << std::endl;
36      std::cin >> numB;
37
38
39      if(greaterThan(numA,numB)){
40          std::cout << "\nNumber A is greater than number B " << std::endl;
41      }
42      else{
43          std::cout << "\nNumber A is NOT greater than number B " << std::endl;
44      };
45
46
47      //Create a function that will take two Boolean values and display the result of all logical operations then return true if it was a success.
48      std::cout << "\nEnter first number: " << std::endl;
49      std::cin >> x;
50
51
52      std::cout << "Enter second number: " << std::endl;
53      std::cin >> y;
54
55
56      std::cout<<operations(x,y);
57
58
59  };
60
61
62  int sumOfTwoNum(int fNum, int sNum){
63      int sum = fNum + sNum;
64      return sum;
65  };
66
67
68  bool greaterThan(int numA, int numB){
69
70      return numA > numB;
```

```
71
72
73  };
74
75
76  bool operations(int x, int y){
77      std::cout<< "\nLogical Operations:"<<std::endl;
78      std::cout<< "\nAND (x && y): "<< (x && y) << std::endl;
79      std::cout<< "OR (x || y): "<< (x || y) << std::endl;
80      std::cout<< "NOT (!x): "<< (!x) << std::endl;
81      std::cout<< "NOT (!y): "<< (!y) << std::endl;
82
83
84      return true;
85  };
```

Sections	Answers
Header File Declaration Section	#include <iostream>
Global Declaration Section	None

Class Declaration and Method Definition Section	int sumOfTwoNum(int fNum, int sNum);
	bool greaterThan(int numA, int numB);
	bool operations(int x, int y);

Main Function	int main()
	{
	
	int fNum, sNum, numA, numB, x, y;
	
	//Create a function that will take two numbers and display the sum.
	
	std::cout << "Enter first number: " << std::endl;
	std::cin >> fNum;
	
	std::cout << "Enter second number: " << std::endl;
	std::cin >> sNum;
	
	std::cout << "\nThe sum of the two number is "<<sumOfTwoNum(fNum,sNum)<< std::endl;
	
	//Create a function that will return whether variable A is greater than variable B.
	std::cout << "\nEnter first number: " << std::endl;
	std::cin >> numA;
	
	std::cout << "Enter second number: " << std::endl;
	std::cin >> numB;
	
	if(greaterThan(numA,numB)){
	std::cout << "\nNumber A is greater than number B " << std::endl;
	}
	else{

	std::cout << "\nNumber A is NOT greater than number B " << std::endl; };

Method Definition	int sumOfTwoNum(int fNum, int sNum){
	int sum = fNum + sNum;
	return sum;
	};
	
	bool greaterThan(int numA, int numB){
	return numA > numB;
	
	};
	
	bool operations(int x, int y){
	std::cout<< "\nLogical Operations:"<<std::endl;
	std::cout<< "\nAND (x && y): "<< (x && y) << std::endl;
	std::cout<< "OR (x || y): "<< (x || y) << std::endl;
	std::cout<< "NOT (!x): "<< (!x) << std::endl;
	std::cout<< "NOT (!y): "<< (!y) << std::endl;
	
	return true;
	};

Table 1-1. C++ Structure Code for Answer

Example 1:

```
Enter first number:
10
Enter second number:
2

The sum of the two number is 12

Enter first number:
5
Enter second number:
2

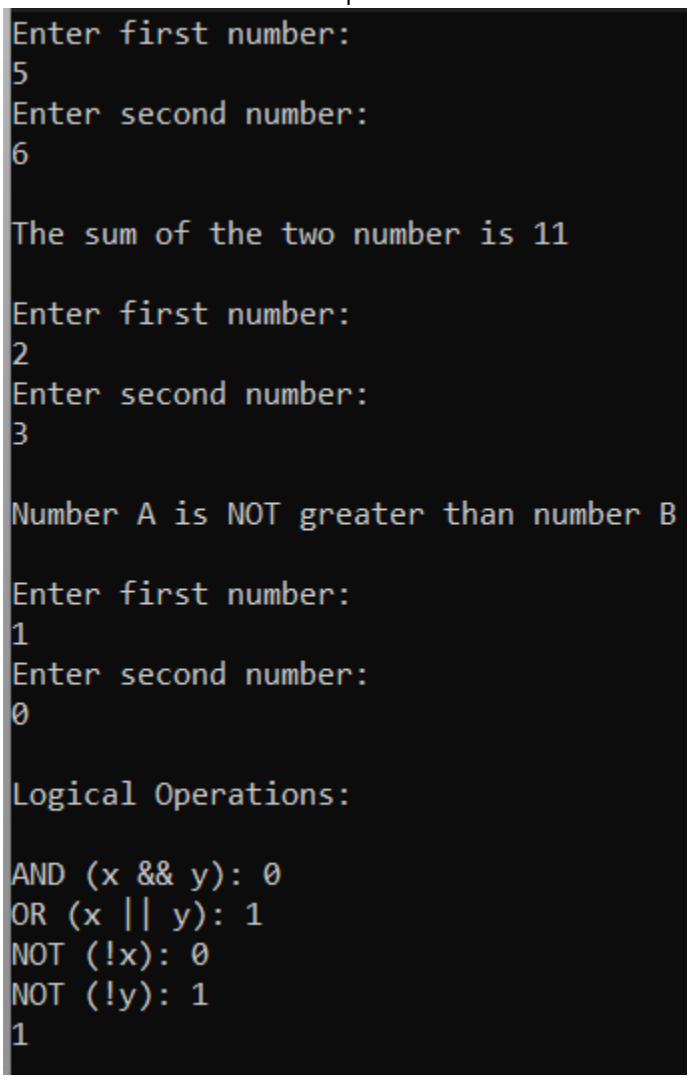
Number A is greater than number B

Enter first number:
2
Enter second number:
3

Logical Operations:

AND (x && y): 1
OR (x || y): 1
NOT (!x): 0
NOT (!y): 0
```

Example 2:



Code Analysis: The program first starts by declaring the function prototypes which are the sumOfTwoNum, greaterThan, and operations. At the end of the code, the definitions for these functions are declared. The sumOfTwoNum function operates by taking the two variables from the parameter and getting their sum. The second function works similarly by taking the two variables in the parameter and comparing if the first variable is greater than the second variable and returning a boolean value (1 or 0). The third function also returns boolean values. It will go through different logical operations which are AND, OR, and NOT logical operations. Each will return a 1 or 0 (true or false). After, the function will return true once successful. The main function technically does the same thing for each of the functions. The program will ask the user for two integers and will call the first function. After, the program will ask again for two integers, the function is then integrated inside an if statement. If the function returns true, the program will display “Number A is greater than number B” and vice-versa. The program will then ask again for two integers, this time, calling the third function that will put the two values in the different logical operations.

Table 1-2. ILO A output observations and comments.

ILO B: Implement Appropriate Class Definition and Instances Based on Given Requirements

In this section, the initial implementation for a class triangle will be implemented.

ILO_B.cpp

```cpp
#include <iostream>


class Triangle{
    private:
        double totalAngle, angleA, angleB, angleC;
    public:
    Triangle(double A, double B, double C);
    void setAngles (double A, double B, double C);
    const bool validateTriangle();


};


int main(){


    Triangle set1(cl);


    if(set1.validateTriangle()){
        std::cout<<"The shape is a valid triangle \n";
    }
    else{
        std::cout<<"The shape is NOT a valid triangle \n";
    }


}


Triangle::Triangle(double A, double B, double C){
    angleA = A;
    angleB = B;
    angleC = C;
    totalAngle = A + B + C;
}
```

```
39
40
41  void Triangle::setAngles(double A, double B, double C){
42      angleA = A;
43      angleB = B;
44      angleC = C;
45      totalAngle = A + B + C;
46  }
47
48
49  const bool Triangle::validateTriangle(){
50      return(totalAngle <= 180);
51  }
```

Sections	Answers
Header File Declaration Section	#include <iostream>
Global Declaration Section	None.

Class Declaration and Method Definition Section	class Triangle{
	private:
	double totalAngle, angleA, angleB, angleC;
	public:
	Triangle(double A, double B, double C);
	void setAngles (double A, double B, double C);
	const bool validateTriangle();
	
	};

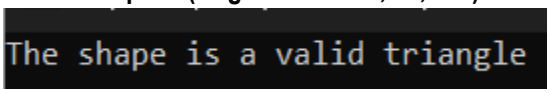
Main Function	int main(){
	
	Triangle set1(40,30,110);
	
	if(set1.validateTriangle()){
	std::cout<<"The shape is a valid triangle \n";
	}
	else{
	std::cout<<"The shape is NOT a valid triangle \n";
	}

	}

Method Definition	Triangle::Triangle(double A, double B, double C){
	angleA = A;
	angleB = B;
	angleC = C;
	totalAngle = A + B + C;
	}
	
	void Triangle::setAngles(double A, double B, double C){
	angleA = A;
	angleB = B;
	angleC = C;
	totalAngle = A + B + C;
	}
	
	const bool Triangle::validateTriangle(){
	return(totalAngle <= 180);
	}

Table 2-1. C++ Structure Code for Answer

Example 1 (Angles used: 40, 30, 110):



Example 2 (Angles used: 90, 90, 90):



Code Analysis: The program starts by declaring the class that is needed. First we assign private variables with the double data type. Then, we create public methods with the first one being the triangle method containing the angle variables and the second is setAngles. The last one is bool data type to return if the triangle is valid or not . We then define the methods after the main function. The triangle function adds the angles to get the totalAngle. The setAngles to change the initial values. The last one validates the triangle by comparing the totalAngle to 180, returning a boolean value. The main function calls the functions with if statements. If the validateTriangle function returns a true value, the program will display “The shape is a valid triangle” and vice-versa.

Table 2-2. ILO B output observations and comments.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILO C: Solve Different Problems using the C++ Programming Language

ILO B Modified.cpp

```cpp
#include <iostream>
#include <cmath>


class Triangle{
    private:
        double totalAngle, angleA, angleB, angleC;
        double sideA, sideB, sideC;
    public:
    Triangle(double A, double B, double C);
    void setAngles (double A, double B, double C);
    void setSides (double a, double b, double c){
        sideA = a;
        sideB = b;
        sideC = c;
    };
    const bool validateTriangle();

    //new functions
    double computeArea();
    double computePerimeter();
    void classifyType();

};
```

```
25
26
27  int main(){
28
29      double angleA, angleB, angleC;
30      double sideA, sideB, sideC;
31
32      std::cout<<"Enter the value of the three angles: "<<std::endl;
33      std::cin>>angleA;
34      std::cin>>angleB;
35      std::cin>>angleC;
36
37      Triangle set1(angleA, angleB, angleC);
38
39
40      if(set1.validateTriangle()){
41          std::cout<<"The shape is a valid triangle \n";
42
43          std::cout<<"\nEnter the value of the three sides: "<<std::endl;
44          std::cin>>sideA;
45          std::cin>>sideB;
46          std::cin>>sideC;
47          set1.setSides(sideA, sideB, sideC);
48
49          std::cout<<"\nArea: "<<set1.computeArea()<<std::endl;
50          std::cout<<"Perimeter: "<<set1.computePerimeter()<<std::endl;
51          std::cout<<"Type: ";
52          set1.classifyType();
53      }
54      else{
55          std::cout<<"The shape is NOT a valid triangle \n";
56      }
```

```
56      }
57
58
59  }
60
61
62  Triangle::Triangle(double A, double B, double C){
63      angleA = A;
64      angleB = B;
65      angleC = C;
66      totalAngle = A + B + C;
67  }
68
69
70  void Triangle::setAngles(double A, double B, double C){
71      angleA = A;
72      angleB = B;
73      angleC = C;
74      totalAngle = A + B + C;
75  }
76
77
78  const bool Triangle::validateTriangle(){
79      return(totalAngle <= 180);
80  }
81
82  double Triangle::computeArea(){
83      double s = (sideA + sideB + sideC) / 2;
84      return sqrt(s * (s - sideA) * (s - sideB) * (s - sideC));
85  }
86
```

```
86
87  double Triangle::computePerimeter(){
88      return sideA + sideB + sideC;
89  }
90
91  void Triangle::classifyType(){
92      if (angleA < 90 && angleB < 90 && angleC < 90)
93          std::cout<<"It is an Acute Triangle."<<std::endl;
94      else if (angleA > 90 || angleB > 90 || angleC > 90)
95          std::cout<<"It is an Obtuse Triangle."<<std::endl;
96      else
97          std::cout<<"Others."<<std::endl;
98  }
```

Sections	Answers
Header File Declaration Section	#include <iostream> #include <cmath>
Global Declaration Section	None.

Class Declaration and Method Definition Section	class Triangle{
	private:
	double totalAngle, angleA, angleB, angleC;
	double sideA, sideB, sideC;
	public:
	Triangle(double A, double B, double C);
	void setAngles (double A, double B, double C);
	void setSides (double a, double b, double c){
	sideA = a;
	sideB = b;
	sideC = c;
	};
	const bool validateTriangle();
	
	//new functions
	double computeArea();
	double computePerimeter();
	void classifyType();
	
	};

Main Function	int main(){
	
	double angleA, angleB, angleC;
	double sideA, sideB, sideC;
	
	std::cout<<"Enter the value of the three angles: "<<std::endl;
	std::cin>>angleA;
	std::cin>>angleB;
	std::cin>>angleC;
	
	Triangle set1(angleA, angleB, angleC);
	
	if(set1.validateTriangle()){
	std::cout<<"The shape is a valid triangle \n";

	std::cout<<"\nEnter the value of the three sides: "<<std::endl;
	std::cin>>sideA;
	std::cin>>sideB;
	std::cin>>sideC;
	set1.setSides(sideA, sideB, sideC);
	
	std::cout<<"\nArea: "<<set1.computeArea()<<std::endl;
	std::cout<<"Perimeter: "<<set1.computePerimeter()<<std::endl;
	std::cout<<"Type: ";
	set1.classifyType();
	}
	else{
	std::cout<<"The shape is NOT a valid triangle \n";
	}
	
	}

Method Definition	Triangle::Triangle(double A, double B, double C){
	angleA = A;
	angleB = B;
	angleC = C;
	totalAngle = A + B + C;
	}
	
	void Triangle::setAngles(double A, double B, double C){
	angleA = A;
	angleB = B;
	angleC = C;
	totalAngle = A + B + C;
	}
	
	const bool Triangle::validateTriangle(){
	return(totalAngle <= 180);
	}
	
	double Triangle::computeArea(){
	double s = (sideA + sideB + sideC) / 2;
	return sqrt(s * (s - sideA) * (s - sideB) * (s - sideC));

```
}

double Triangle::computePerimeter(){
        return sideA + sideB + sideC;
}

void Triangle::classifyType(){
        if (angleA < 90 && angleB < 90 && angleC < 90)
                std::cout<<"It is an Acute Triangle."<<std::endl;
        else if (angleA > 90 || angleB > 90 || angleC > 90)
                std::cout<<"It is an Obtuse Triangle."<<std::endl;
        else
                std::cout<<"Others."<<std::endl;
}
```

Table 3-1. C++ Structure Code for Answer

Output:

```
Enter the value of the three angles:
40
30
110
The shape is a valid triangle

Enter the value of the three sides:
90
90
90

Area: 3507.4
Perimeter: 270
Type: It is an Obtuse Triangle.

--------------------------------
Process exited after 14.47 seconds with return value 0
Press any key to continue . . .
```

Code Analysis: First, I modified private and public classes to include variables for the sides. After, I declared the different methods that were asked in the question. These are the computeArea, computePerimeter, and classifyType. Then, I defined each method after the main function. First is the computeArea method which first

calculates the semi-perimeter needed to compute the actual area. After, we use the function square root from the cmath library along with the formula for the area of a triangle. It will then return the calculated value. Next is the computePerimeter which returns the calculated value by adding all of the sides. The third method defined is the classifyPerimeter which uses if and else if statements. The first if statement uses the AND logical operator to check if each angle is less than 90, displaying that it is an acute triangle. The else if statement uses the OR logical operator and checks if each angle is greater than 90 and displays that it is an obtuse. The else statement displays others. Finally, the main function first declares variables for the three angles and sides. Then, the program allows the user to input the values of the angles. After checking if the angles form a valid triangle, the program will proceed to ask the user for three sides. The program will call each function we created to display the area, perimeter, and type. If it is not a valid triangle, the program will display that it is not.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **ChatGPT**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Terms explanation**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **I used AI to help me recall some terminologies such as functions and methods.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.